

# Longhorn Toe-Dip

**install → one replicated model volume → serve from two GPUs → host access → teardown**

A full-lifecycle plan to try Longhorn **without migrating anything**. One model — **Qwen3.6-27B NVFP4** (~15 GB, NVIDIA's repo on Hugging Face; exact repo id confirmed at download time) — downloaded **once**, served by vLLM on **a2 and a3 at the same time**, and visible on the host filesystem for vanilla Docker / venv vLLM. Then we break it a little, and tear it down. Nothing is installed yet — this is the plan to review before executing. Companion theory: docs/13. Tracked as Forgejo **brian/home-lab#27**; feeds benchmark issue **#7**.

**Status** plan only — nothing installed

**Replica disks** a2 + a3 — CMR 6 TB HDDs, never a root

**Untouched** a1 · spark · every existing PVC

**Issues** #27 (this) → #7 (benchmark)

**1×**

## DOWNLOAD

~15 GB pulled once into one PVC — never again per node

**2**

## GPUS AT ONCE

vLLM on a2 and a3 mount the same volume simultaneously

**2**

## REPLICAS

Live copies on 2 nodes — survives a dead disk

**0**

## MIGRATIONS

Grafana, Prometheus, everything else stays exactly where it is

## The big question, answered up front

"Can two GPUs serve one downloaded copy, and can I still reach the files from plain Docker or a venv on the host?" — **YES**.

A Longhorn **RWX volume** (backed by Longhorn's internal NFS *share-manager*) lets multiple pods on different nodes mount the same filesystem at the same time — and because it's just NFS behind a ClusterIP, the **host can mount it too**, at `/mnt/models`, and point `HF_HOME` there. Same bytes for k8s pods, vanilla Docker, and venv vLLM. Zero re-downloads.

### Honest nuance — read before you fall in love

The files are **not physically copied to every node's disk**. They live on **2 replicas** and are **served over the LAN** to everyone else. Two different mechanisms working together here: **replication** = durability + mobility (2 live copies, docs/13 p.1–2); **RWX-NFS** = simultaneous access (one NFS server in front of the volume). This plan deliberately uses both.

### Speed check (so nothing surprises you)

The replicas live on **spinning HDDs** (p.2) — and that costs nothing here: these WD Blues read sequentially at ~150–180 MB/s, while real-world WiFi moves ~25–50 MB/s — and tonight **every node is on WiFi** (p.3). So the network is the bottleneck either way: loading 15 GB ≈ **5–10 min** vs ~30 s from local NVMe. Fine for this test; docs/13 p.3 explains when *not* to use RWX for weights. Model load time ≠ inference speed — once weights are in VRAM, tokens/sec is identical.

## Guardrails — what this is **NOT**

- **Not a migration.** Grafana, Prometheus, Harbor, Forgejo, Paperless — every existing PVC stays on `local-path`, untouched.
- **Longhorn is NOT made the default StorageClass.** Nothing new lands on it by accident.
- **No backups yet.** Backup target + schedules are issues **#18/#21**, after this experiment.
- **a1 sits out the toe-dip** — fewer variables. But it's no longer "excluded, period": its 6 TB ext4 drive makes it a plausible *later* bulk member (p.3).
- **Nothing touches spark** — arm64, other subnet (192.168.6.x), often offline. Pure compute, as decided in docs/13 p.2.

## The real drive inventory — audited over SSH

Models verified via `lsblk` (MODEL + rotational flag). The headline: the cluster's bulk storage is **spinning HDDs** — and the two this plan uses are the right kind (**CMR**, see below).

| NODE | DRIVE                       | KIND              | FS / MOUNT                  | FREE  | LONGHORN ROLE                             |
|------|-----------------------------|-------------------|-----------------------------|-------|-------------------------------------------|
| a2   | Samsung 980 PRO 2 TB        | NVMe              | ext4 · root                 | 463 G | root — protected · later NVMe pool (p.7)  |
| a2   | WD Blue 6 TB<br>WD60EZAX    | HDD · CMR         | ext4 · /home/brian/e        | 5.3 T | Longhorn disk                             |
| a2   | WD Blue 4 TB<br>WD40EZAZ    | HDD · SMR         | NTFS · "d"                  | —     | future capacity (reformat ext4, p.7)      |
| a2   | WD_BLACK SN850X 2 TB        | NVMe              | NTFS                        | —     | future capacity (reformat ext4, p.7)      |
| a3   | Samsung 850 EVO 250 GB      | SATA SSD          | ext4 · root                 | 70 G  | never tiny root — permanently out         |
| a3   | WD Blue 6 TB<br>WD60EZAX    | HDD · CMR         | ext4 · /mnt/d (2.6 T part.) | 1.8 T | Longhorn disk (2.9 T NTFS part. = future) |
| x1   | 476 G NVMe (LUKS)           | NVMe              | encrypted root              | 242 G | no disk — components only (p.3)           |
| a1   | Kingston A400 120 G         | SATA SSD          | root                        | 46 G  | never tiny root                           |
| a1   | WD Blue 6 TB<br>WD60EZAX    | HDD · SMR         | ext4 · /mnt/d               | 4.8 T | optional bulk member, later (p.3)         |
| a1   | WD Black 1 TB<br>WD1003FZEX | HDD · CMR<br>7200 | unmounted                   | —     | unused today                              |

### SMR vs CMR — the one thing to check on any spinning drive

**CMR** (conventional) writes every track independently — sustained random writes and full-replica rebuilds are fine. **SMR** (shingled) overlaps tracks like roof tiles: sequential writes are OK, but sustained random writes — and exactly the **full-replica rebuilds** Longhorn does — collapse to a crawl once the drive's CMR-style cache zone fills. This is the infamous WD Red NAS debacle. Decoder for this fleet: **WD60EZAX = CMR** — luckily **both** drives the plan uses (the a2 + a3 6 TB) are this one; **WD60EZAZ / WD40EZAZ = SMR** (a1's big drive, a2's NTFS 4 TB) — fine for read-mostly bulk, last pick for anything write-hot.

## Where Longhorn's disks live — never on a root partition

**Principle:** Longhorn never touches a root partition in this plan. Left alone, it puts its disk at `/var/lib/longhorn` — on the root filesystem. We override that with per-node disk config pointing at the big ext4 CMR HDDs:

```
# disk config comes from an annotation instead of the /var/lib/longhorn default:
kubectl label node a2 a3 node.longhorn.io/create-default-disk=config
kubectl annotate node a2 node.longhorn.io/default-disks-config=\
'["path":"/home/brian/e/longhorn","allowScheduling":true]}'
kubectl annotate node a3 node.longhorn.io/default-disks-config=\
'["path":"/mnt/d/longhorn","allowScheduling":true]}'
```

### ext4 is all it needs — no reformat, no dedicated partition

A Longhorn "disk" is just a directory: replicas are plain **sparse files on the existing ext4 filesystem**, coexisting with whatever already lives on the drive (Docker data, downloads). Only the NTFS drives are unusable until reformatted.

## Member roles — a deterministic replica-2 layout

| NODE       | LONGHORN ROLE       | DETAIL                                                                                            |
|------------|---------------------|---------------------------------------------------------------------------------------------------|
| a2         | disk + components   | Replica home #1 — 5.3 T free CMR HDD. Runs one vLLM pod.                                          |
| a3         | disk + components   | Replica home #2 — 1.8 T free CMR HDD. Runs the other vLLM pod.                                    |
| x1         | components, NO disk | Can attach/mount volumes; holds no data. Its only drive is the LUKS NVMe <b>root</b> — protected. |
| a1 / spark | out                 | a1: out of the toe-dip, optional later (below). spark: never — arch/subnet/uptime.                |

Disks on exactly two nodes  $\Rightarrow$  every replica-2 volume lands **one copy on a2's HDD, one on a3's**. Deterministic — no scheduler surprises, no "which node got the replica?"

### a1, revisited — optional bulk member, last in line

For **write-once/read-many bulk volumes** (model files — exactly what its 4.8 T-free ext4 drive could hold), a1 is plausible later: no real-time write path to wait on, and if a replica there rots, Longhorn rebuilds elsewhere and the workload moves. Honest caveats: its USB WiFi makes it the most likely node to *trigger* rebuild churn, and its HDD is **SMR** — rebuilds to and from it are slow. Verdict: **after the pattern is proven, not during it**.

**Network audit note:** nobody is wired — a2 eno2, a3 eno2, x1 enp0s31f6 are all NO-CARRIER; every member rides WiFi. Fine here: the one heavy event is the ~15 GB download mirroring to replica 2 — tens of minutes, once. Ethernet stays the biggest replication-*performance* lever (p.7) — a lever, not a gate.

## Prereqs on a2, a3, x1 (run on each host)

**What we do:** install the iSCSI initiator (volumes attach as iSCSI block devices) + NFS client (RWX mounts). The audit confirmed both are **missing on all three** — required everywhere, not a maybe.

```
sudo apt-get update && sudo apt-get install -y open-iscsi nfs-common
sudo systemctl enable --now iscsid
# preflight check (version must match the chart you install):
curl -sSfL https://raw.githubusercontent.com/longhorn/\
longhorn/v1.9.0/scripts/environment_check.sh | bash
# multipathd (the usual Ubuntu gotcha): verified inactive on all three — nothing to do.
```

## Install Longhorn — a Helm release, not kubectl

**What we do:** label the 3 members, then `helm install latest stable 1.x` into `longhorn-system`, with committed values that fence every Longhorn component onto `a2/a3/x1`. **Why Helm:** same rule as Homepage in CLAUDE.md — the chart is a managed release; applying rendered YAML with `kubectl` creates namespace drift and fights future upgrades.

```
# components fence — a2/a3/x1. (Disk placement was configured on p.3: a2 + a3 only.)
kubectl label node a2 a3 x1 inference-club.com/longhorn=member

helm repo add longhorn https://charts.longhorn.io && helm repo update
helm install longhorn longhorn/longhorn \
  -n longhorn-system --create-namespace \
  -f clusters/home/longhorn/values.yaml      # values are committed to the repo
```

### clusters/home/longhorn/values.yaml — the four decisions

```
defaultSettings:
  defaultReplicaCount: 2                # 2 live copies per volume
  createDefaultDiskLabeledNodes: true   # honor p.3's labels — never /var/lib/longhorn
  systemManagedComponentsNodeSelector: "inference-club.com/longhorn:member"
persistence:
  defaultClass: false                   # longhorn SC exists but is NOT default
longhornManager:
  nodeSelector: { inference-club.com/longhorn: member }
longhornUI:
  nodeSelector: { inference-club.com/longhorn: member }
```

Both selectors matter: `longhornManager` covers the `DaemonSet`, `systemManagedComponentsNodeSelector` covers everything Longhorn spawns itself (instance-managers, share-managers, CSI pods). Result: **nothing lands on a1 or spark**.

## UI at https://longhorn.lan

Standard pattern: Traefik ingress + `home-tls` wildcard cert + `gethomepage.dev/*` annotations so it appears on Homepage — same as every other `.lan` service. Ingress manifest lives next to the values file.

### The Longhorn UI ships with NO auth — and it can delete volumes

Anyone who can reach `longhorn.lan` can detach or delete volumes with two clicks. Acceptable on the LAN for a toe-dip; ☐ **follow-up:** Traefik `basic-auth` middleware before anything precious lives on Longhorn. Do not expose over Tailscale.

### Verify the install before moving on

- UI → **Node** tab: 3 nodes running components; **disks only on a2 + a3**, at `/home/brian/e/longhorn` and `/mnt/d/longhorn` — and **nothing at /var/lib/longhorn**. `x1` shows diskless (attach-only).
- `kubectl -n longhorn-system get pods -o wide | grep -E 'a1|spark' → empty`.
- `kubectl get sc → longhorn` exists, `local-path` still marked (default).

## One RWX volume, one download, two vLLM servers

**What we do:** a new service dir `services/qwen36-nvfp4/` with three pieces — a PVC, a one-shot download Job, and two vLLM Deployments. **Why:** the PVC's `ReadWriteMany` mode is the whole trick — creating it makes Longhorn spin up a **share-manager** pod (the NFS server) automatically. No NFS to configure by hand.

### (a) The PVC

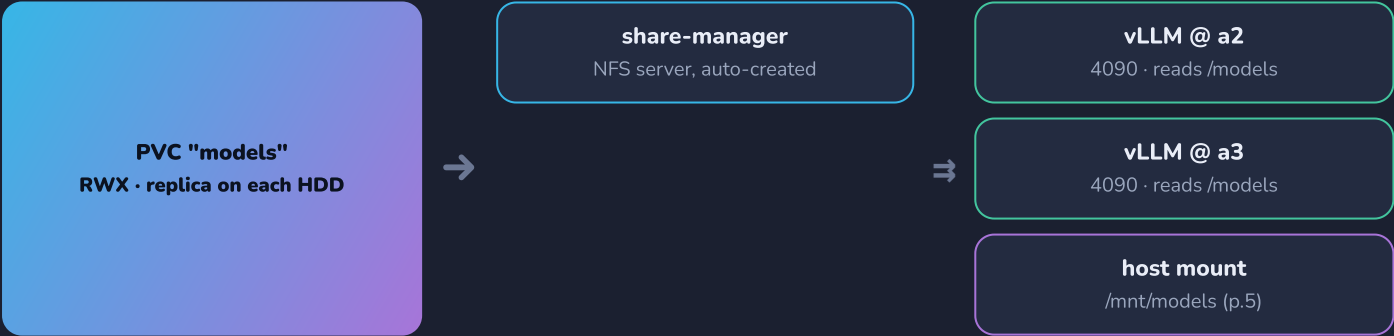
```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: models
spec:
  accessModes: [ReadWriteMany] # ← RWX
  storageClassName: longhorn
  resources:
    requests: { storage: 40Gi }
```

### (b) The download Job — once, on a2

```
spec:
  template:
    spec:
      # a2 = the 5.3 T HDD; never a1
      nodeSelector:
        inference-club.com/box: a2
      containers:
        - name: fetch
          image: python:3.12-slim
          env: [{ name: HF_HOME, value: /models }]
          command: ["bash", "-lc",
            "pip install -U 'huggingface_hub[cli]' &&
            hf download nvidia/Qwen3.6-27B-NVFP4"]
          # ↑ confirm exact repo id at download time
```

### (c) Two vLLM Deployments — same volume, different GPUs

| FIELD        | QWEN-A2                                                                       | QWEN-A3                    |
|--------------|-------------------------------------------------------------------------------|----------------------------|
| nodeSelector | inference-club.com/box: a2                                                    | inference-club.com/box: a3 |
| GPU          | runtimeClassName: nvidia · nvidia.com/gpu: 1 (exclusive, per GPU conventions) |                            |
| Volume       | PVC models at /models, readOnly: true · HF_HOME=/models                       |                            |
| API          | OpenAI-compatible on :8000 behind ClusterIP Services qwen-a2 / qwen-a3        |                            |



**NVFP4-on-Ada caveat (docs/13 p.4 · issue #7)**

The 4090 is **Ada**; NVFP4 kernels prefer **Blackwell**. If vLLM refuses the format on the 4090s, **the storage pattern is still proven** — swap in any FP8/AWQ model and the plan is unchanged. Either way, register both endpoints behind **LiteLLM** later for the #7 A/B benchmark.

## Mount the same volume on the host — step by step

**What we do:** find the NFS endpoint Longhorn created for the RWX volume, then NFS-mount it on a2's host at `/mnt/models`.

**Why it works:** the share-manager Service is a ClusterIP, and **kube-proxy makes ClusterIPs reachable from every cluster node's host** — no NodePort, no ingress, no credentials.

```
# 1 · find the share-manager Service for the volume (one per RWX PVC):
kubectl -n longhorn-system get svc | grep pvc-
#   pvc-1a2b3c... ClusterIP 10.43.x.x 2049/TCP

# 2 · on a2's host — mount it:
sudo mkdir -p /mnt/models
sudo mount -t nfs4 -o vers=4.1 <cluster-ip>:/<pvc-name> /mnt/models

# 3 · same files the pods see:
ls /mnt/models/hub/
```

## The two vanilla flows — zero re-download

### venv vLLM on the host

```
HF_HOME=/mnt/models vllm serve \
  nvidia/Qwen3.6-27B-NVFP4 \
  --port 8001
```

vLLM finds the weights already cached under `$HF_HOME/hub` and skips the download entirely.

### plain Docker on the host

```
docker run --gpus all -p 8001:8000 \
  -v /mnt/models:/root/.cache/huggingface \
  vllm/vllm-openai:latest \
  --model nvidia/Qwen3.6-27B-NVFP4
```

`/root/.cache/huggingface` is the default `HF_HOME` inside the image — the bind-mount lands the cache exactly where it looks.

### Caveats — know these before you rely on the mount

- **umount before deleting the volume.** If the PVC goes away under a live mount, the mount hangs hard.
- **The ClusterIP changes** if the PVC is ever recreated — re-run step 1, don't hardcode it in `fstab`.
- **Reads are network-speed** — today that means WiFi (p.3), which is slower than the HDDs themselves (p.1). Fine for weight loading.
- **Only works ON cluster nodes** (kube-proxy routes ClusterIPs there) — which is where the GPUs are anyway.

### Why this matters

This closes the gap between "cluster storage" and "my normal tools." The same 15 GB serves: two k8s vLLM pods, a venv experiment, and a throwaway Docker container — downloaded once, replicated twice, mountable anywhere on the cluster's hosts.

## Verify — three checks, then one brave one

```
# 1 · both GPUs serving the same volume, simultaneously (run from any cluster node):
curl -s http://qwen-a2.qwen36-nvfp4.svc:8000/v1/completions -H 'Content-Type: application/json' \
  -d '{"model":"nvidia/Qwen3.6-27B-NVFP4","prompt":"hello","max_tokens":16}' &
curl -s http://qwen-a3.qwen36-nvfp4.svc:8000/v1/completions -H 'Content-Type: application/json' \
  -d '{"model":"nvidia/Qwen3.6-27B-NVFP4","prompt":"hello","max_tokens":16}' & wait
# 2 · Longhorn UI: volume Healthy, 2 replicas — one on a2's HDD, one on a3's.
# 3 · same bytes, pod vs host mount — hashes must match:
F='hub/models--nvidia-*/snapshots/*/model-00001*.safetensors'
kubect1 -n qwen36-nvfp4 exec deploy/qwen-a3 -- sh -c "sha256sum /models/$F"
sha256sum /mnt/models/$F # on a2's host — hashes must be identical
```

### Optional 5-minute chaos drill — the whole point of Longhorn

In the UI, open the volume and **delete one replica**. Watch Longhorn auto-rebuild it on a healthy node while both vLLM pods keep serving. That's the durability story from docs/13 p.2, demonstrated instead of believed.

## Teardown — in this exact order

```
# 1 · host mount FIRST (see p.5 caveat):
sudo umount /mnt/models
# 2 · the servers:
kubect1 -n qwen36-nvfp4 delete deploy qwen-a2 qwen-a3
```

**Decision point — the PVC.** If **#7 benchmarking continues**, keep it: the weights stay downloaded, replicated, and remountable in seconds. If done, delete it — `kubect1 -n qwen36-nvfp4 delete pvc models` — which **reclaims the space on both replicas at once**. Longhorn itself stays installed either way; it costs little idle.

## Staging the rollout — where this goes

| STAGE   | MOVE                                                                                                                                                                                               |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1·NOW   | <b>This doc:</b> toe-dip on the two CMR HDDs (a2 + a3), full lifecycle, nothing else moves.                                                                                                        |
| 2·NEXT  | The bulk-HDD pool becomes the home for big <b>read-mostly</b> data — models, media.                                                                                                                |
| 3·LATER | Small-fast <b>NVMe pool</b> : a2 980 PRO (463 G free) + x1 NVMe (242 G free), hard <b>storageReserved</b> caps so roots never fill. a3's 70-G-free root SSD stays permanently out.                 |
| 4·OPT   | a1's 4.8 T-free SMR drive joins the bulk pool — read-mostly replicas only, last in line (p.3).                                                                                                     |
| 5·OPT   | Capacity: reformat the NTFS drives to ext4 — a2's 4 TB SMR + SN850X 2 TB, a3's 2.9 T partition.                                                                                                    |
| 6·EXP   | Small <b>Postgres</b> on Longhorn, <b>measured</b> — every write waits for WiFi RTT + a remote HDD seek. An experiment, not a fear: <b>backups (#18/#21) stay the databases' real safety net</b> . |
| 7·ANY   | Ethernet for a2/a3/x1 — the biggest single replication-performance lever, whenever convenient.                                                                                                     |

### Bottom line

Components fenced to a2/a3/x1, **replicas on the two CMR 6 TB HDDs — never a root partition**. One 40 Gi RWX volume, one 15 GB download → served by two 4090s at once *and* mounted on the host — then torn down cleanly, every mechanism of the wider rollout proven. Nothing existing moves, nothing becomes default, a1 and spark never notice.